0.1 Thực nghiệm với mô hình SVM

Trong phần này, chúng ta sẽ đi sâu vào quá trình triển khai mô hình SVM để phân loại bộ dữ liệu MNIST, từ khâu xử lý dữ liệu thô đến việc thiết lập cấu hình huấn luyện tối ưu.

0.1.1 Tiền xử lý dữ liệu (Preprocessing)

Dữ liệu hình ảnh từ bộ MNIST cần được đưa về định dạng phù hợp trước khi đưa vào mô hình SVM. Quy trình thực hiện gồm các bước chính sau:

- **Chuẩn hóa (Normalization):** Các giá trị pixel trong ảnh gốc nằm trong khoảng $[0, 255]$. Chúng ta thực hiện chuẩn hóa về đoạn $[0.0, 1.0]$ bằng công thức:

$$X_{norm} = \frac{X_{raw}}{255.0} \tag{1}$$

  Việc chuẩn hóa giúp các giá trị đặc trưng có cùng quy mô, hỗ trợ hàm Kernel RBF tính toán khoảng cách chính xác hơn và tăng tốc độ hội tụ của thuật toán.

- **Làm phẳng dữ liệu (Flattening):** SVM yêu cầu đầu vào là các vector đặc trưng một chiều. Do đó, mỗi ma trận ảnh kích thước 28×28 được trải phẳng thành một vector có độ dài 784:

$$X \in \mathbb{R}^{28\times 28} \xrightarrow{\text{Flatten}} X' \in \mathbb{R}^{784} \tag{2}$$

- **Quản lý nhãn:** Nhãn dữ liệu được giữ nguyên dưới dạng số nguyên từ 0 đến 9 để phục vụ cho chiến lược phân loại đa lớp.

0.1.2 SVM trong phân loại đa lớp

Về bản chất toán học, thuật toán SVM được thiết kế để giải quyết bài toán phân loại nhị phân (Binary Classification) — tức là chỉ phân biệt giữa hai lớp dữ liệu dựa trên một siêu phẳng tối ưu. Tuy nhiên, bộ dữ liệu MNIST có 10 lớp chữ số từ 0 đến 9. Để giải quyết vấn đề này, thư viện `sklearn.svm.SVC` áp dụng chiến lược **One-vs-One (OvO)**.

Cơ chế hoạt động của chiến lược One-vs-One: Trong chiến lược OvO, thay vì xây dựng một bộ phân loại duy nhất cho 10 lớp, hệ thống sẽ xây dựng các bộ phân loại nhị phân cho từng cặp lớp có thể có.

Số lượng bộ phân loại nhị phân cần thiết được tính theo công thức tổ hợp:

$$N_{classifiers} = \frac{k(k-1)}{2} \tag{3}$$

Trong đó k là số lượng lớp. Với bài toán MNIST ($k = 10$), số lượng bộ phân loại được tạo ra là:

$$N_{classifiers} = \frac{10(10-1)}{2} = 45 \text{ bộ phân loại nhị phân} \tag{4}$$

Cách ra quyết định của AI (Cơ chế bỏ phiếu - Voting): Khi một hình ảnh chữ số mới (vector X') được đưa vào hệ thống để dự đoán, quá trình ra quyết định diễn ra như sau:

1. **Dự đoán song song:** Hình ảnh này sẽ được đưa qua toàn bộ 45 bộ phân loại nhị phân đã huấn luyện.
2. **Tích lũy phiếu bầu:** Mỗi bộ phân loại sẽ đưa ra một "phiếu bầu" cho lớp mà nó cho là đúng nhất.
   - Ví dụ: Bộ phân loại "3 vs 8" nhận định ảnh là số 3, thì lớp "3" sẽ được cộng thêm 1 điểm.
3. **Quyết định cuối cùng:** Lớp nào nhận được nhiều phiếu bầu nhất từ tổng số 45 dự đoán sẽ được AI chọn làm nhãn dự đoán cuối cùng cho hình ảnh đó.

0.1.3 Triển khai (Implement)

a, Sơ đồ cấu trúc thư mục (Project Structure)

Dự án phân loại chữ số MNIST sử dụng SVM được tổ chức theo cấu trúc thư mục sau để đảm bảo tính mô đun và dễ bảo trì:

- **main.py:** File chính để chạy chương trình.
- **Data_preparation/:** Thư mục chứa các module xử lý dữ liệu (data_loader.py).
- **Models/:** Thư mục chứa module svm_model.py.
- **Results/:** Thư mục lưu trữ kết quả và mô hình (svm_model.joblib, Result SVM.txt).

b, Quá trình huấn luyện AI

Quá trình huấn luyện mô hình SVM được thực hiện theo các bước chi tiết sau:

1. **Tải và tiền xử lý dữ liệu:**

- Chuẩn hóa giá trị pixel từ khoảng [0, 255] về [0.0, 1.0].
- Làm phẳng mỗi ảnh 28×28 thành vector 784 chiều.
- Giữ nguyên nhãn dưới dạng số nguyên từ 0 đến 9.

2. **Huấn luyện mô hình:** Mô hình SVM được cấu hình với Kernel RBF, C=1.0, gamma='scale'.
3. **Lưu trữ mô hình:** Mô hình đã huấn luyện được lưu vào file Results/svm_model.joblib.

Các hàm huấn luyện, lưu trữ và đánh giá mô hình được triển khai trong module svm_model.py và được gọi thông qua hàm run_svm_project().

0.2 Thực nghiệm với mô hình CNN

Trong phần này, chúng ta sẽ khám phá việc áp dụng Mạng Nơ-ron Tích chập (CNN) để phân loại bộ dữ liệu MNIST. CNN tận dụng cấu trúc không gian của dữ liệu hình ảnh, mang lại hiệu suất vượt trội so với các phương pháp truyền thống như SVM.

0.2.1 Tiền xử lý dữ liệu (Preprocessing)

Dữ liệu hình ảnh MNIST cần được chuẩn bị phù hợp cho mạng CNN. Quy trình xử lý gồm các bước sau:

- **Chuẩn hóa (Normalization):** Các giá trị pixel từ $[0, 255]$ được chuẩn hóa về $[0.0, 1.0]$ bằng công thức:

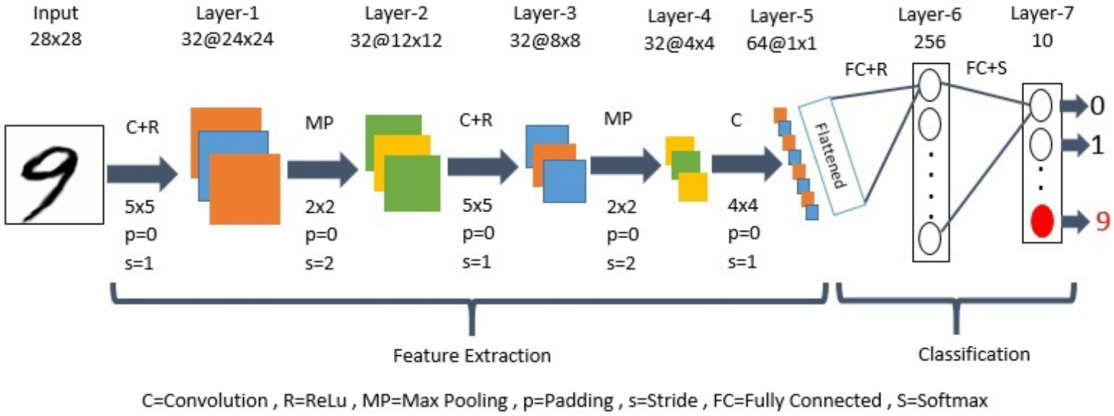
$$X_{norm} = \frac{X_{raw}}{255.0} \tag{5}$$

- **Reshape dữ liệu:** Dữ liệu được định hình lại thành tensor 3 chiều $(28 \times 28 \times 1)$ để phù hợp với đầu vào của CNN:

$$X \in \mathbb{R}^{784} \xrightarrow{\text{Reshape}} X' \in \mathbb{R}^{28\times28\times1} \tag{6}$$

- **Mã hóa nhãn (One-hot encoding):** Nhãn được chuyển đổi từ số nguyên sang vector one-hot.

0.2.2 Thiết kế kiến trúc mạng sequential



Hình 0.1: Minh họa kiến trúc mạng sequential trong mô hình CNN.

Bảng 1: Chi tiết cấu trúc các tầng trong mô hình CNN đề xuất

Tầng (Layer)	Tham số cấu hình	Chức năng
Conv2D_1	32 filters, size 3x3, ReLU	Trích xuất các đặc trưng cơ bản (cạnh, nét).
MaxPooling_1	Pool size 2x2	Giảm kích thước không gian, giữ đặc trưng mạnh nhất.
Conv2D_2	64 filters, size 3x3, ReLU	Trích xuất các đặc trưng phức tạp hơn.
MaxPooling_2	Pool size 2x2	Tiếp tục giảm chiều dữ liệu.
Conv2D_3	64 filters, size 3x3, ReLU	Trích xuất đặc trưng bậc cao.
Flatten	-	Chuyển đổi Tensor 3D thành Vector 1D.
Dense (Hidden)	64 units, ReLU	Kết hợp các đặc trưng để chuẩn bị phân loại.
Dense (Output)	10 units, Softmax	Trả về xác suất dự đoán cho 10 chữ số.

0.2.3 Triển khai (Implement)

a, Sơ đồ cấu trúc thư mục (Project Structure)

Dự án phân loại chữ số MNIST sử dụng CNN được tổ chức theo cấu trúc thư mục tương tự như mô hình SVM:

- **main.py:** File chính để chạy chương trình.
- **Data_preparation/:** Thư mục chứa module data_loader.py.
- **Models/:** Thư mục chứa module cnn_model.py.
- **Results/:** Thư mục lưu trữ kết quả và mô hình (mnist_cnn_model.h5, Result CNN.txt).

b, Quá trình huấn luyện AI

Quá trình huấn luyện mô hình CNN được thực hiện theo các bước chi tiết sau:

1. **Tải và tiền xử lý dữ liệu:**
   - Reshape dữ liệu thành (28, 28, 1).
   - Chuẩn hóa giá trị pixel về [0.0, 1.0].
   - Mã hóa nhãn thành dạng one-hot.
2. **Xây dựng kiến trúc mạng:** Mô hình bao gồm 3 lớp tích chập với MaxPooling, Flatten, và 2 lớp Dense. Sử dụng Optimizer Adam, Loss Categorical Crossentropy.
3. **Huấn luyện mô hình:** Với epochs: 5, Batch size: 64, Validation split: 10%.
4. **Lưu trữ mô hình:** Mô hình được lưu vào file Results/mnist_cnn_model.h5.

Các hàm xây dựng, huấn luyện, lưu trữ và đánh giá mô hình được triển khai trong module cnn_model.py và được gọi thông qua hàm run_cnn_project().